

Feature Engineering for Trading with Initial Balance and Greeks

1. Project Context

I am developing an algorithmic trading system that combines:

- **Initial Balance (IB)**: Price structure of the opening hours of the market
 - **Options Greeks**: Critical levels of gamma, delta, vanna, charm, vomma, vega, etc.
 - **Machine Learning**: Neural network (MLP + Attention) for predictions
 - The main bots for this neural network are located in `bots/collect_training_data.py`, `bots/hybrid_model.py`, `bots/train_hybrid.py`, `bots/visualize_hybrid.py`, `bots/trading_wrapper.py`, `bots/backtest_hybrid.py`, `bots/data_utils.py`, `bots/tradingbot_wrapper.py`.
-

2. Adding Vega and Vomma to Features

Study the files in `services/gexdaemon.py` to see which Greeks are currently calculated, and how they are handled in `bots/collect_training_data.py`. I want you to add **Vega** and **Vomma** using the Black-Scholes model.

Available Data in `option_data` DataFrame

You have EVERYTHING needed to calculate EXACT Vega and Vomma. Your `option_data` DataFrame contains:

- ✓ `strike_price` : Strike price
- ✓ `call_iv`, `put_iv` : Implied volatility per strike
- ✓ `call_gamma`, `put_gamma` : Individual gamma per contract
- ✓ `call_delta`, `put_delta` : Individual delta
- ✓ `call_open_int`, `put_open_int` : Open Interest (contract volume)
- ✓ `time_till_exp` : Time to expiration in years
- ✓ `spot_price` : Available in your global context

Conclusion: You can calculate Vega and Vomma per strike and then aggregate them just like you do with Gamma.

Black-Scholes Exact Formulas

Vega per Strike:

```
Vega_unit = S * exp(-q*T) * sqrt(T) * norm.pdf(d1)
```

Vomma per Strike:

```
Vomma_unit = Vega_unit * (d1 * d2) / sigma
```

Where:

- S = spot_price
- K = strike_price
- T = time_till_exp (time to expiration in years)
- σ = implied volatility (call_iv or put_iv)
- $r = 0.03$ (risk-free rate)
- $q = 0.0$ (dividend yield, use 0.0 for SPX)
- $d1 = [\ln(S/K) + (r - q + 0.5\sigma^2)*T] / (\sigma\sqrt{T})$
- $d2 = d1 - \sigma\sqrt{T}$

Total Exposure (Dealer Exposure)

Transform unit values into Dealer Exposure by multiplying by Open Interest and 100 (standard multiplier):

```
Vega_Exposure = Vega_unit * Open_Interest * 100 * direction
Vomma_Exposure = Vomma_unit * Open_Interest * 100 * direction
# direction = +1 for Calls (dealer short), -1 for Puts (dealer long)
```

Aggregate these exposures to get `totalvega` and find the strike with highest exposure for `max_vega`.

IMPORTANT: The sign convention must match your current GEX logic:

Gamma measures how much an option's delta changes due to a change in the underlying asset's price. Odte-gamma measures the gamma exposure for a 1% move in asset price.

Long (+) gamma: Hedging goes against the market (buy the dip, sell rallies), which reduces volatility and stabilizes the market. Strikes with large + exposures are 'walls' that push price away to nearest significant - exposure.

Short (-) gamma: Hedging goes with the market (selling more as price falls / buying more as it moves up), which brings volatility and destabilizes the market. Strikes with large - exposures are 'walls' but often are price magnets.

For example, if total gamma was 1.2 billion, that amount would be sold for a 1% move up and bought for a 1% move down. If it was -1.2 billion, that amount would be sold for a 1% move down and bought for a 1% move up.

ODTE vs. Weekly Greeks Considerations

Critical Note: Since you are trading ODTE options, as time (T) approaches zero, **Vega and Vomma tend to ZERO.**

In ODTE, **Gamma, Vanna, and Charm** are the absolute kings of intraday flow. Vega and Vomma have minimal impact at 2:00 PM on expiration day. However, the model also reads Weekly Greeks (e.g., `wk_net_gamma`). Injecting Vega and Vomma from weekly expirations provides a perfect map of whether institutions are scared or relaxed at the macro level.

In your system:

- Variables with `odte_` prefix: Current day Greeks (Gamma, Charm dominant)
- Variables with `wk_` prefix: Weekly expiration Greeks (Vega, Vomma relevant)

For Vega/Vomma calculation:

1. Calculate separately for ODTE and Weekly contracts
2. Generate variables:
 - `odte_totalvega`, `odte_max_vega`, `odte_min_vega` (expected: very low values)
 - `wk_totalvega`, `wk_max_vega`, `wk_min_vega` (significant values)
 - Same pattern for Vomma

Add to CSV created by `collect_training_data`:

- Vega: `totalvega`, `max_vega`, `min_vega`
- Vomma: `totalvomma`, `max_vomma`, `min_vomma`

In the Hybrid neural network model, add features to combine Vega and Vomma with different Greeks, as you have with Gamma, DGEX, and other Greeks.

3. Core Objective

Create a **Confluence Matrix** that cross-references:

1. Initial Balance structure with Fibonacci extensions
 2. Maximum and minimum levels of Greeks
 3. Normalized distances to detect high-probability zones
-

4. Required Features

4.1. Initial Balance Extensions

Calculate Fibonacci levels from the IB:

Bullish extensions (above IB High):

- `fib_127_up` : $IB_High + (0.272 \times IB_Range)$
- `fib_161_up` : $IB_High + (0.618 \times IB_Range)$
- `fib_200_up` : $IB_High + (1.0 \times IB_Range)$

Bearish extensions (below IB Low):

- `fib_127_down` : $IB_Low - (0.272 \times IB_Range)$
- `fib_161_down` : $IB_Low - (0.618 \times IB_Range)$
- `fib_200_down` : $IB_Low - (1.0 \times IB_Range)$

4.2. Normalized Distances (Critical for ML)

All distances must be **relative to IB_Range** to avoid scale problems:

```
dist_normalized = (Target_Level - Current_Price) / IB_Range
```

A. Distances from Current Price to Fibonacci Extensions:

- `dist_price_to_fib_127_up`, `dist_price_to_fib_161_up`,
`dist_price_to_fib_200_up`
- `dist_price_to_fib_127_down`, `dist_price_to_fib_161_down`,
`dist_price_to_fib_200_down`

B. Distances IB High vs Maximum Greeks:

- `dist_ib_high_vs_max_gamma`
- `dist_ib_high_vs_max_vanna`
- `dist_ib_high_vs_max_charm`
- `dist_ib_high_vs_max_zomma`
- `dist_ib_high_vs_max_vega`
- `dist_ib_high_vs_max_vomma`

C. Distances IB Low vs Minimum Greeks:

- `dist_ib_low_vs_min_gamma`
- `dist_ib_low_vs_min_vanna`
- `dist_ib_low_vs_min_charm`
- `dist_ib_low_vs_min_zomma`
- `dist_ib_low_vs_min_vega`
- `dist_ib_low_vs_min_vomma`

D. Distances IB vs GEX Support Greeks (Negatives):

- `dist_ib_low_vs_max_dgex`
- `dist_ib_high_vs_max_dgex`
- `dist_ib_high_vs_min_dgex`
- `dist_ib_low_vs_min_dgex`

E. Distances Fibonacci Extensions vs Greeks (Most Powerful):

For EACH Fibonacci extension (6 levels) cross with EACH Greek (7 Greeks):

- **Extensions:** `fib_127_up`, `fib_161_up`, `fib_200_up`, `fib_127_down`, `fib_161_down`, `fib_200_down`
- **Greeks:** `max_gamma`, `min_gamma`, `max_vanna`, `min_vanna`, `max_charm`, `min_charm`, `max_dgex`, `max_zomma`, `min_zomma`, `max_vega`, `min_vega`, `max_vomma`, `min_vomma`

Total: 6 extensions × 13 Greek levels = 78 confluence distances

Examples:

- `dist_fib_161_up_vs_max_gamma`
- `dist_fib_161_down_vs_max_dgex`
- `dist_fib_200_up_vs_max_vega`
- `dist_fib_127_down_vs_min_vomma`
- ... (all combinations)

Note: All calculated as `abs(level_a - level_b) / ib_range`

Study `services/gexdaemon.py` to see the data generated in JSON. Add all relevant IB vs Greeks combinations you identify.

4.3. Historical Context

Z-Score of IB Range:

- `ib_range_zscore_5d` : Z-Score of the IB range compared to last 5 days

Formula:

```
ib_range_zscore_5d = (IB_Range_today - mean_5d) / std_5d
```

Interpretation:

- Indicates whether the market is compressed or expanded relative to recent history
- Values > 2.0: Abnormally large range (expansion)
- Values < -1.0: Compressed range (accumulation)
- Values $\approx$ 0: Normal range

Important Notes:

- We only have about two dozen days of data
- First 5 days will have `ib_range_zscore_5d = NaN` (incomplete window)
- Your model must discard rows with NaN in Z-Score, or fill them with 0.0

4.4. Confluences with RBF Kernel (Signals)

Continuous **floating variables** that measure proximity between levels using a Gaussian function.

Why RBF Kernel is Superior:

Your MLP likely uses ReLU, tanh, or sigmoid - all are non-linear activation functions.

- If you pass **linear features**, you force the network to learn non-linearity during training (wasting capacity)
- If you pass **RBF features** (already non-linear), layers focus on learning confluence combinations, not transforming distances

Formula:

```
# Value between 0.0 (far) and 1.0 (touching) using RBF kernel
distance = abs(level_a - level_b) / ib_range
confluence = np.exp(-distance**2 / (2 * sigma**2))
```

Sigma (σ) Hyperparameter:

- **σ too small (0.01):** Only near-perfect confluences receive value 1.0 → Network too restrictive
- **σ too large (0.20):** Even large distances receive high values → Loss of selectivity

Recommended Configuration for Daytrading:

- `sigma = 0.05` (5% of IB_Range)
- Since this is daytrading, only confluences < 3-5 points are relevant for SPX or /ES, on SPY and QQQ and /NQ the equivalent would be 0.05% distance of the price up or down
- Values `confluence > 0.37` indicate critical reaction zone
- Values `confluence < 0.10` are considered practically irrelevant

Interpretation:

- `confluence = 1.0` : Levels coincide exactly
- `confluence = 0.92` : Very close (within ~2% of IB_Range)
- `confluence = 0.37` : Moderately close (~5% of IB_Range)
- `confluence < 0.10` : No relevant confluence

Examples of Critical Confluences:

- `confluence_high_gamma` : IB High vs maximum gamma (institutional resistance)
- `confluence_low_dgex` : IB Low vs negative GEX (institutional support)
- `confluence_fib161up_gamma` : Extension 1.618 up vs critical vanna (bullish target)
- `confluence_fib161down_gamma` : Extension 1.618 down vs maximum gamma (bearish target)

IMPORTANT: Calculate confluences for:

- IB High/Low vs ALL Greeks (max and min)
- EACH Fibonacci extension vs ALL Greeks
- **Total: 100+ confluence variables**

5. Dimensionality Reduction (CRITICAL)

With 100+ confluence variables, there is a **severe risk of overfitting**. Implement this 3-step hybrid reduction strategy:

```
# =====  
# STEP 1: Variance Filtering (Conservative)
```

```

# =====
# Eliminates variables that NEVER activate (variance < 0.01)
variances = df[confluence_cols].var()
high_variance = variances > 0.01
useful_confs = confluence_cols[high_variance] # Remaining ~60 out of 100

def select_top_confluences(df, confluence_cols, top_n=30):
    """
    Selects the top N confluences with highest cross-temporal variance.
    Keeps only those that actually fluctuate (carry signal).

    Args:
        df: DataFrame with all confluence columns
        confluence_cols: List of confluence column names
        top_n: Number of top confluences to keep (default 30)

    Returns:
        DataFrame with only top N confluence columns
        List of selected column names
    """
    variances = df[confluence_cols].var()
    top_cols = variances.nlargest(top_n).index.tolist()

    print(f"Selected {len(top_cols)} confluences out of {len(confluence_cols)}")
    print(f"Variance range: {variances[top_cols].min():.4f} - {variances[top_cols].max():.4f}")

    return df[top_cols], top_cols

# =====
# STEP 2: Correlation Analysis
# =====
# Identify highly correlated groups (e.g., all fib_161 vs gamma/vanna/cha
corr_matrix = df[useful_confs].corr()

# Find groups with correlation > 0.8
high_corr_groups = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.8:
            high_corr_groups.append((corr_matrix.columns[i], corr_matrix.columns[j]))

print(f"Found {len(high_corr_groups)} highly correlated pairs")

# =====

```



```

# STEP 3: Grouped PCA (ONLY on correlated groups)
# =====
# Apply PCA only to groups of highly correlated features
# Example configuration (adjust based on your correlation analysis):

from sklearn.decomposition import PCA

# Group A: fib_161_up vs all Greeks (10 variables) → 3 components
fib161_up_cols = [col for col in useful_confs if 'fib_161_up' in col]
if len(fib161_up_cols) > 3:
    pca_a = PCA(n_components=3)
    pca_a_result = pca_a.fit_transform(df[fib161_up_cols])
    df_pca_a = pd.DataFrame(
        pca_a_result,
        columns=[f'pca_fib161up_pc{i+1}' for i in range(3)]
    )
    print(f"Group A explained variance: {pca_a.explained_variance_ratio_}")

# Group B: ib_high vs Greeks (6 variables) → 2 components
ib_high_cols = [col for col in useful_confs if 'ib_high_vs' in col]
if len(ib_high_cols) > 2:
    pca_b = PCA(n_components=2)
    pca_b_result = pca_b.fit_transform(df[ib_high_cols])
    df_pca_b = pd.DataFrame(
        pca_b_result,
        columns=[f'pca_ibhigh_pc{i+1}' for i in range(2)]
    )
    print(f"Group B explained variance: {pca_b.explained_variance_ratio_}")

# Rest: Keep original features that are not highly correlated
# Final result: ~5 PCA components + ~45 original confluences = ~50 features

```

When to Apply: After calculating all confluences, right before saving the CSV.

Criterion: `top_n=30` (adjustable based on cross-validation results)

6. Implementation Instructions

Study files in `bots/` and `services/` folders to understand how the neural network works and how data is saved.

I need you to write **complete Python code** for:

6.1. Function to Calculate All Features

Function that receives a DataFrame with:

- Initial Balance columns: `ib_high`, `ib_low`, `ib_range`
- Current price: `close` or `price`
- Maximum/Minimum Greeks: `max_gamma`, `min_gamma`, `max_vanna`, `min_vanna`, `max_charm`, `min_charm`, `max_dgex`, `min_dgex`, `max_zomma`, `min_zomma`, `max_vega`, `min_vega`, `max_vomma`, `min_vomma`

6.2. Function Must Return

The function must return expanded DataFrame with:

- All calculated Fibonacci extensions (6 levels)
- All normalized distances (~84 distance features)
- 5-day Z-Score of IB Range
- All confluence variables (100+ RBF signals)
- After dimensionality reduction: ~30-50 final features

6.3. Pipeline Integration

Show how to:

1. Call this function inside the data collection loop
 2. Calculate Z-Score at the end, when all days have been processed
 3. Apply dimensionality reduction before saving CSV
 4. Generate final list of `FEATURE_COLUMNS` for the model
-

7. Important Considerations

Avoid Collinearity:

- Do NOT include `price`, `ib_high` AND `price_vs_ib_high` (they are redundant)
- The normalized distances already capture this relationship

Prevent Overfitting:

- Normalize everything so the network learns patterns, not absolute values
- Use dimensionality reduction to keep only high-signal features
- With only ~20 days of data, be conservative with feature count

Confluences are Triggers:

- When AI sees `confluence ≈ 1.0` + price approaching → Can predict violent movements
 - These are the most powerful predictive features in the system
-

8. Edge Cases and Validations

Cases to Handle:

1. `IB_Range = 0` : Replace with `np.nan` or `1.0` (minimum point)

```
ib_range = df['ib_range'].replace(0, np.nan)
```

2. **Greeks with NaN** : Use `.fillna(0)` after calculation

```
df[greek_cols] = df[greek_cols].fillna(0)
```

3. `std_5d = 0` : Replace with `np.nan` in Z-Score calculation

```
std_5d = df['ib_range'].rolling(5).std().replace(0, np.nan)
```

4. **Strikes without OI** : Vega/Vomma will be 0 automatically (correct behavior)

5. **Confluences with inf** : Replace with 0

```
df[confluence_cols] = df[confluence_cols].replace([np.inf, -np.inf],
```

Post-Calculation Validations:

```
# Verify no infinite values
assert not df[confluence_cols].isin([np.inf, -np.inf]).any().any(), \
    "Found infinite values in confluences"

# Verify confluence range (0.0 - 1.0)
assert (df[confluence_cols] >= 0).all().all(), \
    "Found negative confluence values"
assert (df[confluence_cols] <= 1.0).all().all(), \
    "Found confluence values > 1.0"
```

```
# Verify normalized distances are reasonable
dist_cols = [col for col in df.columns if col.startswith('dist_')]
assert (df[dist_cols].abs() < 10).all().all(), \
    "Found unreasonable normalized distances (>10 IB_Ranges)"

# Verify Z-Score is reasonable (should be between -3 and 3 most of the ti
zscore_col = 'ib_range_zscore_5d'
extreme_zscores = (df[zscore_col].abs() > 3).sum()
if extreme_zscores > len(df) * 0.05: # More than 5% extreme values
    print(f"WARNING: {extreme_zscores} extreme Z-scores found (>5% of dat
```

9. Expected Output

Production-ready Python code that I can copy directly into my system, containing:

9.1. Core Functions

- `calculate_vega_vomma_by_strike(option_data, spot_price)` : Black-Scholes calculation
- `aggregate_vega_vomma(option_data)` : Aggregate by Open Interest
- `calculate_fibonacci_extensions(df)` : Calculate 6 Fib levels
- `calculate_normalized_distances(df)` : All 84 distance features
- `calculate_rbf_confluences(df, sigma=0.05)` : All 100+ confluences
- `calculate_zscore_5d(df)` : Rolling Z-Score
- `apply_dimensionality_reduction(df, confluence_cols)` : 3-step reduction

9.2. Documentation

- Clear docstrings for all functions
- Inline comments explaining logic
- Examples of expected input/output
- Handling of edge cases documented
- When you finish all the implementation, update `docs/FEATURES.md` and `docs/HYBRID_MODEL.md` and `README.md`

9.3. Integration Code

- Complete list of `FEATURE_COLUMNS` to add to model configuration

9.4. Validation Suite

- All validation checks from Section 8
- Verification that output matches expected format